Comparing Programming Languages

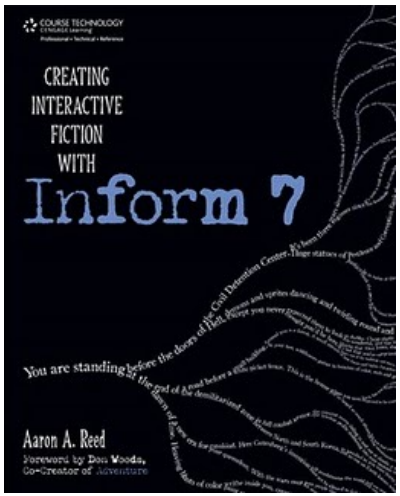
James Gain
Department of Computer Science
University of Cape Town
jgain@cs.uct.ac.za

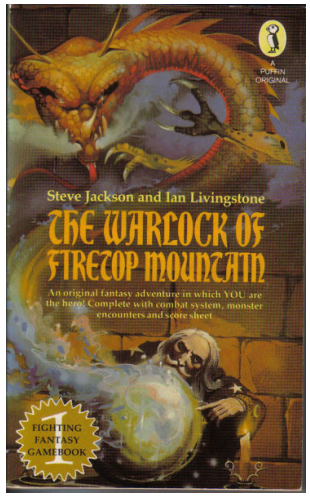




Introduction

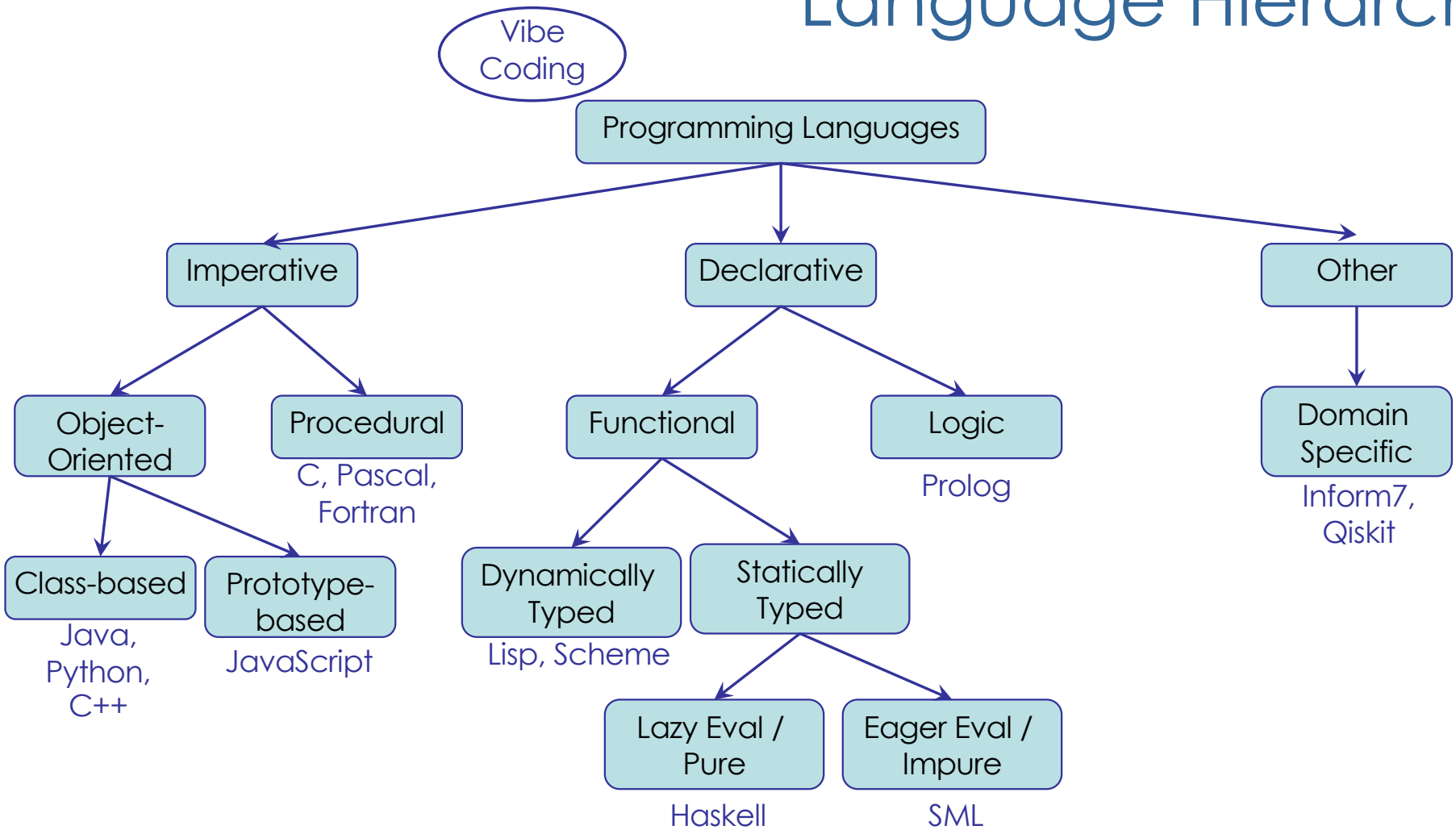
- Take a step back to place Functional Programming in context
- Compare programming paradigms
  - Fundamental concepts
  - Style that embodies philosophy (e.g., Pythonic coding)
  - Benefits
- Domain-specific languages
  - Inform7 (for Interactive Fiction games)
  - Qiskit (for Quantum Programming)





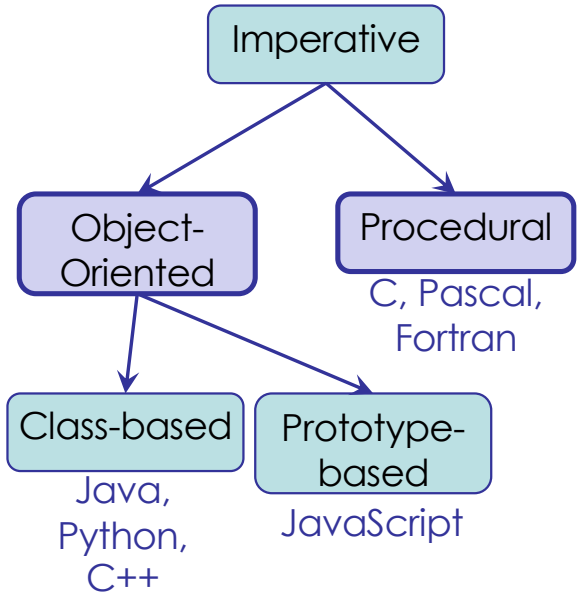
Qiskit

Language Hierarchy



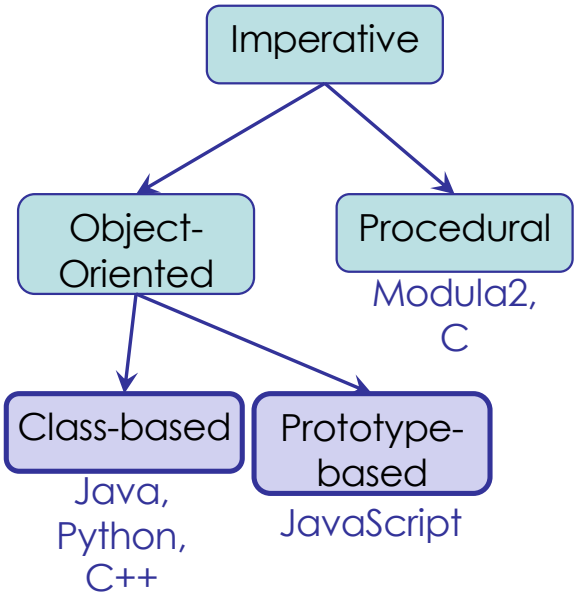
Imperative Programming

- Procedural
  - An early approach, that organizes code sequentially based on procedures (functions)
  - Benefits – abstraction, simplicity, performance
  - Unlike FP, allows mutability, explicit looping and tracks state change
- Object-oriented
  - Data and functionality encapsulated together in objects
  - Benefits – code reusability (through inheritance and polymorphism), modularity, security (through encapsulation), abstraction



Object-Oriented Variants

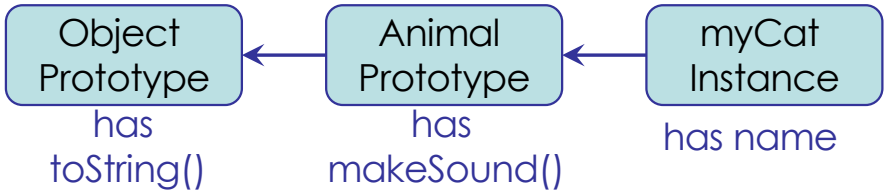
- Class-based
  - A class is a static blueprint specified at compile time that is instantiated by an object with specific data
  - Subclasses inherit from a superclass
- Prototype-based
  - Objects are cloned and extended dynamically, potentially at run time, from template objects with no concept of a class
  - (Note - not the same as templates in C++)
  - Objects link to a prototype chain through delegation



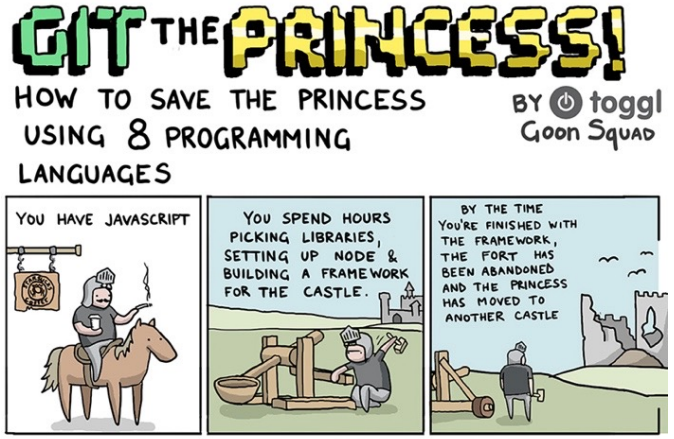
Example: JavaScript

- Prototype:
  - Objects point to a chain of other objects that define behaviour
  - Calling a property or method involves walking up the chain to find the definition
- Mutable:
  - Objects in the prototype chain can be altered at run-time (e.g., `Object.setPrototypeOf(obj, newProto)`)
  - A new object can be added to the front of the chain

```
1  // Define a constructor function
2  function Animal(name) {
3    this.name = name;
4  }
5
6  // Add a method to the Animal prototype
7  Animal.prototype.makeSound = function() {
8    return `${this.name} makes a sound.`;
9  };
10
11 // Create an instance of Animal
12 const myCat = new Animal("Nia");
13
14 // Access a property and a method
15 console.log(myCat.name);
16 console.log(myCat.makeSound());
17 console.log(myCat.toString()); // Output: "[object Object]"
```



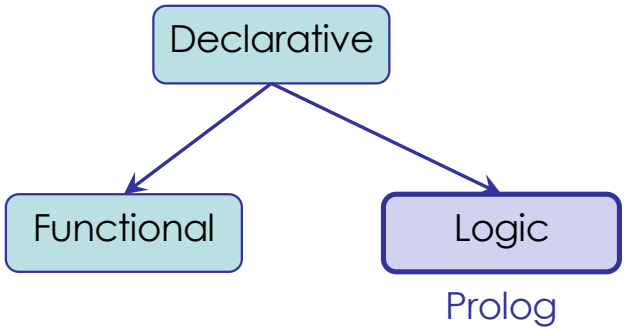
The World's Worst Programming Language



- Rushed early development
  - Created for Netscape Navigator in 10 days
  - To allow client-side form validation and basic animations by non-coder
- Flaws baked in
  - Backwards compatible so that legacy web scripts remain functional
- Web monopoly
  - Browsers only natively executed HTML, CSS & JavaScript
- Example
  - By default, sort treats numbers as strings, so '10' comes before '2' lexicographically

Logic Programming

- A form of declarative programming built on formal logic
  - Facts – unconditional statements
  - Rules – logical relationships of the form `head :- body` where `head` is true if `body` is true
  - Queries – used to infer answers from the defined facts and rules
- Benefits
  - Separation of logic and control
  - Reversibility – a definition can often be run both ways
- Use cases
  - Expert systems, automated theorem proving, natural language processing



Prolog Example: Penguins

```
1  % --- FACTS ---
2  has_feathers(tweety).
3  has_feathers(kowalski).
4  can_swim(kowalski).
5
6  % --- RULES ---
7  % An animal is a bird if it has feathers.
8  is_bird(X) :-
9      has_feathers(X).
10
11 % An animal is a penguin if it is a bird and can swim.
12 is_penguin(X) :-
13     is_bird(X),
14     can_swim(X).
15
16 % Birds can usually fly, unless they are a penguin!
17 can_fly(X) :-
18     is_bird(X),
19     \+ is_penguin(X).   % "\+" means "NOT" in Prolog
```

SYNTAX

Atoms are enclosed in single quotes or start with a small letter

Read as: IF

Every clause is terminated by a '.'

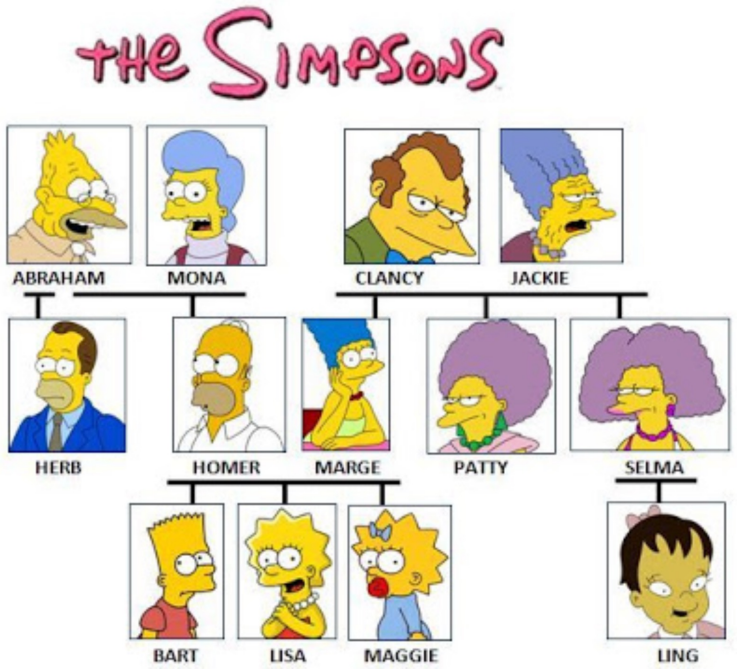
Variables begin with a capital or underscore

',' means AND. ';' means OR

Prolog Example: Lineage

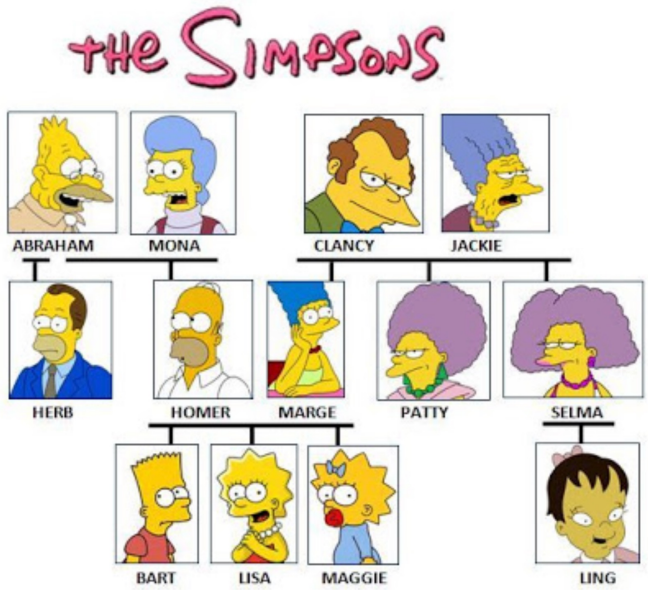
```
1  % --- FACTS ---
2  male(bart).
3  male(homer).
4  male(herb).
5  male(abraham).
6  male(clancy).
7
8  female(lisa).
9  female(maggie).
10 female(marge).
11 female(mona).
12 female(jackie).
13
14 % Parent relationships: parent(Parent, Child)
15 parent(marge, bart).
16 parent(marge, lisa).
```

ETC



Prolog Example: Lineage

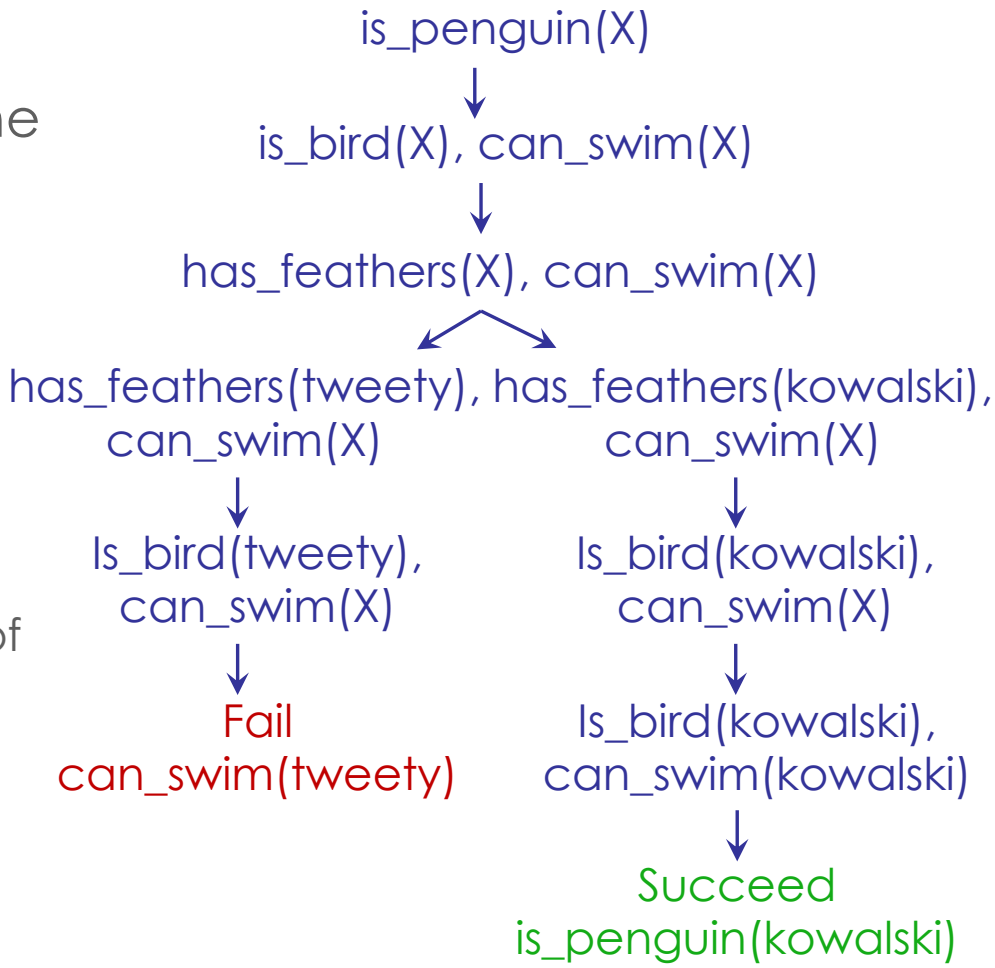
```
28 % --- RULES ---
29
30 % X is the father of Y if X is the parent of Y and X is male.
31 father(X, Y) :-
32     parent(X, Y),
33     male(X).
34
35 % X is the mother of Y if X is the parent of Y and X is female.
36 mother(X, Y) :-
37     parent(X, Y),
38     female(X).
39
40 % X and Y are siblings if they share the same parent and X is not Y.
41 sibling(X, Y) :-
42     parent(Z, X),
43     parent(Z, Y),
44     X \= Y.
45
46 % X is the grandmother of Y if X is the parent of Z, and Z is the parent of Y, and X is female.
47 grandmother(X, Y) :-
48     parent(X, Z),
49     parent(Z, Y),
50     female(X).
```



ETC

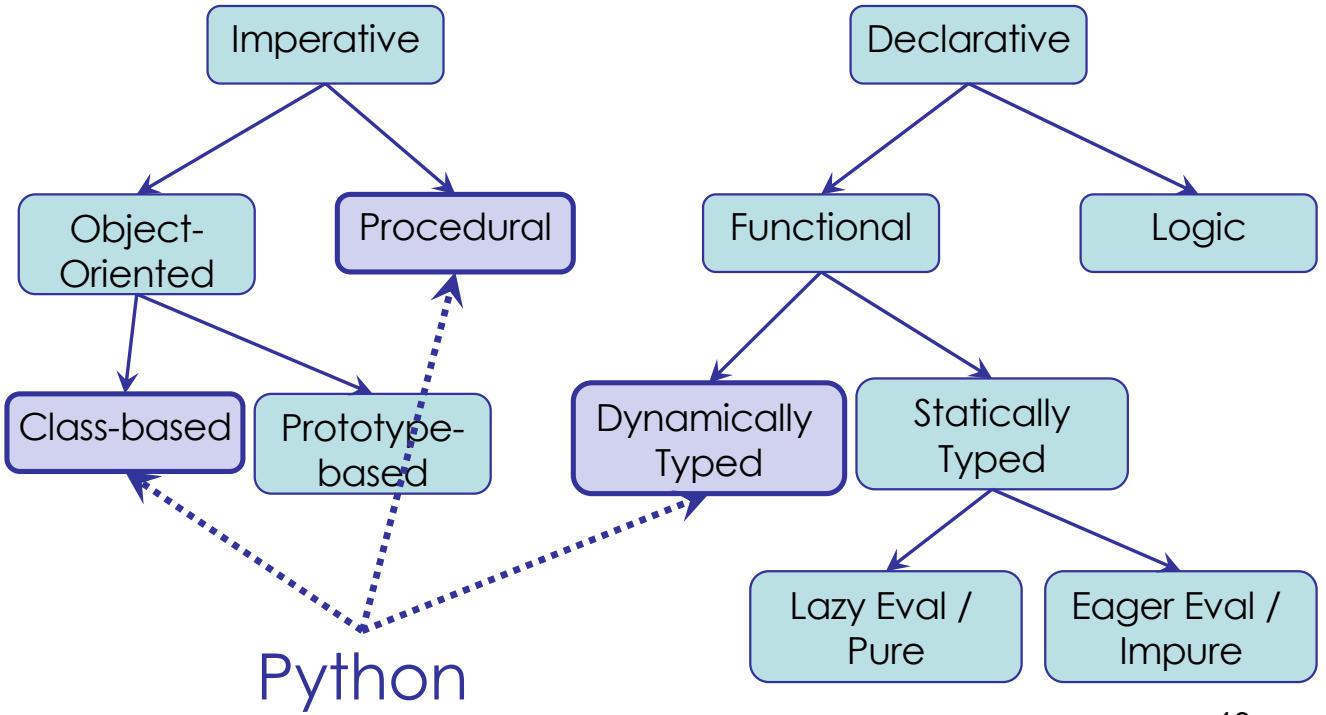
Prolog Solution Mechanism

- Prolog uses a depth-first search of the query state space
  - Employing unification, resolution, and backtracking
- Process for a subgoal
  - Look for first matching clause
  - Unification – attempts to instantiate variables
  - Resolution – unifying leads to either proof or generation of more subgoals
- Depth-first search with backtracking
  - Explores a subgoal as deeply
  - On failure returns to a previous choice



Languages cross boundaries

- Several languages pick and choose elements
  - Don't fit a single category
- Python
  - Class-based object-oriented
  - Procedural
  - Functional (lambda functions)



Summary

- A broad divide between Imperative and Declarative Programming
  - But some languages (e.g., Python, C++) span the divide
- Logic programming is declarative and built on formal logic
  - Answers queries using facts and rules